2025 IEEE International Conference on Robotics and Automation (ICRA)
May 19-23, 2025. Atlanta, USA

X-MOBILITY: End-To-End Generalizable Navigation via World Modeling

Wei Liu[1*], Huihua Zhao[1*], Chenran Li[1,2], Joydeep Biswas[1]
Billy Okal[1], Pulkit Goyal[1], Yan Chang[1], Soha Pouya[1]

***Abstract*— General-purpose navigation in challenging environments remains a significant problem in robotics, with current state-of-the-art approaches facing myriad limitations. Classical approaches struggle with cluttered settings and require extensive tuning, while learning-based methods face difficulties generalizing to out-of-distribution environments. This paper introduces X-MOBILITY, an end-to-end generalizable navigation model that overcomes existing challenges by leveraging three key ideas. First, X-MOBILITY employs an auto-regressive world modeling architecture with a latent state space to capture world dynamics. Second, a diverse set of multi-head decoders enables the model to learn a rich state representation that correlates strongly with effective navigation skills. Third, by decoupling world modeling from action policy, our architecture can train effectively on a variety of data sources, both with and without expert policies—off-policy data allows the model to learn world dynamics, while on-policy data with supervisory control enables optimal action policy learning. Through extensive experiments, we demonstrate that X-MOBILITY not only generalizes effectively but also surpasses current state-of-the-art navigation approaches. Additionally, X-MOBILITY also achieves zero-shot Sim2Real transferability and shows strong potential for cross-embodiment generalization.**

Project page: https://nvlabs.github.io/X-MOBILITY

I. INTRODUCTION

Developing a versatile navigation stack for robots is challenging due to the need for adaptability across diverse environments and robot platforms. Such a system must adjust to varying operational constraints while maintaining high efficiency, particularly when operating with limited onboard computational resources. Classical approaches [1], [2] often rely on modular architectures, integrating separate navigation components to meet specific needs. Although this modular design offers some generalization, it typically requires complex configurations and extensive tuning for each new deployment [3]. Moreover, these methods often struggle to manage uncertainties in both the environment and the robot's state, which can compromise system robustness. To address this limitation, the Partially Observable Markov Decision Process (POMDP) [4] framework has been proposed, which models probabilistic belief states and solve decision-making problems by interleaving observation and action. However, POMDPs demand significant system modeling effort [5], limiting scalability, and their computational intensity poses challenges for real-time deployment on edge devices [6].

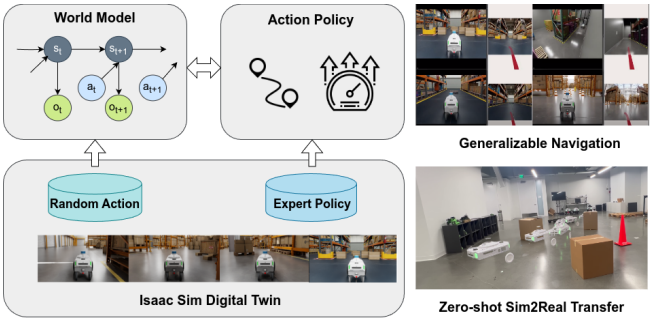

Fig. 1. X-MOBILITY: an end-to-end world model based navigation stack featuring a multi-stage training pipeline using photorealistic synthetic datasets, demonstrating generalizability across out-of-distribution scenarios and zero-shot Sim2Real transferability.

In response to these challenges, end-to-end learning-based navigation stacks have emerged as a promising alternative [7]–[9]. These systems leverage experience to optimize navigation performance and generalize across various tasks. Toward this end, reinforcement learning has been widely explored in robotics, but its application to navigation has been limited by issues such as low sampling efficiency in large-scale environments and difficulties in scaling simulations [10]. Imitation Learning (IL) presents an alternative by enabling robots to learn directly from demonstrations, which has been particularly successful in fields where large-scale datasets with teacher policies are readily available [11]. However, IL often encounters the problem of covariate shift, where performance deteriorates in scenarios outside the training data distribution [12]. To mitigate this, methods such as DAgger [13] have been proposed, which iteratively expand the training dataset by incorporating the model's errors and the expert's corrections. Additionally, recent advancements in world modeling [14], which aim to learn world dynamics in both in-distribution and out-of-distribution environments, have shown promise in enhancing robustness, particularly in autonomous driving [15]–[17]. However, a key challenge in applying these approaches to robot navigation is the scarcity of large-scale datasets, especially those needed for imitation learning.

Recognizing these challenges, this paper introduces X-MOBILITY, an end-to-end navigation model designed to generalize across various mobility applications (see Fig. 1). Inspired by world modeling, X-MOBILITY employs a lightweight auto-regressive learning architecture to develop a rich latent representation space that probabilistically captures the world state and its dynamics. This representation space is effectively trained and strongly correlated with robust

[*] Equal contribution

[1] Wei Liu, Huihua Zhao, Chenran Li, Joydeep Biswas, Billy Okal, Pulkit Goyal, Yan Chang, and Soha Pouya are with NVIDIA, Santa Clara, California, USA {liuw, huihuaz, chenranl, jbiswas, bokal, pulkitg, yachang, spouya}@nvidia.com

[2] Chenran Li is also with Department of Mechanical Engineering, UC Berkeley, Berkeley, California, USA. The work is conducted during Chenran Li's internship at NVIDIA chenran_li@berkeley.edu

2025 IEEE International Conference on Robotics and Automation (ICRA) | 979-8-3315-4139-2/25/$31.00 ©2025 IEEE | DOI: 10.1109/ICRA55743.2025.11128692

navigation skills through a set of diverse multi-task decoders. To address data scarcity, X-MOBILITY decouples world modeling from action policy imitation, allowing it to train from a variety of data sources, both with and without supervisory control inputs: off-policy data sources enable the model to learn world dynamics, while on-policy sources with supervisory control facilitate the action policy learning.

With this architectural design, X-MOBILITY is trained through a multi-stage pipeline that leverages NVIDIA's Isaac Sim to generate large-scale, photorealistic synthetic datasets featuring diverse scenes and action policies. Extensive experiments demonstrate that X-MOBILITY can consistently outperform current state-of-the-art methods and exhibits zero-shot Sim2Real transferability, showing good potential for generalization across different robot embodiments. The key contributions of this paper are:

- Introduction of an end-to-end, world model-based navigation model that outperforms state-of-the-art methods and demonstrates zero-shot mobility in challenging environments not present in the training data.
- Design of a model architecture with decoupled world modeling and action policy networks, enabling efficient training from a wide range of data sources, thus addressing data scarcity challenges in robot navigation.
- Validation of the feasibility of training the navigation model with photorealistic synthetic datasets, achieving zero-shot Sim2Real transferability.

The remainder of this paper is structured as follows: Section II presents the X-MOBILITY model architecture and its component design. Section III details the experimental setup, and the results are discussed in Section IV. Section V discusses the Sim2Real transfer and cross-embodiment generalization, and the paper concludes in Section VI.

II. X-MOBILITY

A. Problem Formulation

The general-purpose robot navigation problem can be formulated as a POMDP, defined by the tuple $\{\mathcal{S}, \mathcal{A}, \mathcal{O}, T, O, R, \gamma\}$ consisting of the state space $\mathcal{S}$, action space $\mathcal{A}$, the observation space $\mathcal{O}$, transition function $T(s', s, a) = \Pr(s'|s, a)$, and observation function $O(o, s', a) = \Pr(o|s', a)$. The reward function $\mathrm{R}(s, a)$ defines the reward for performing action a in state s, and $\gamma \in [0, 1)$ is the discount factor. The solution to the POMDP is an optimal policy π^* that maximizes the expected accumulated reward, $E\left(\sum_{t=0}^{\infty} \gamma^t \mathrm{R}(a_t, s_t)\right)$, where s_t and a_t represent the agent's state and action at time t.

X-MOBILITY addresses the challenge of generalizable navigation by solving the POMDP within a learned state representation space, which includes a state estimation network, a state prediction network, and an action policy network as in Fig. 2. The state estimation and prediction networks focus on learning world dynamics, while the action policy network aims to solve the POMDP by imitating the teacher policy, assuming it closely approximates the optimal policy π^*.

B. Belief Representation In Learned State Space

In classical robot navigation, the state space is often manually defined, such as $\mathbb{SE}(2)$ for the robot's location. Previous efforts have enriched state representations by incorporating semantic features like tracked obstacles [18] or terrain characteristics [19], but defining a sufficiently expressive and computationally tractable state space for generalizable navigation remains a challenge.

Instead of manually defining the state space, X-MOBILITY uses a learned state representation to capture all relevant navigation features. To handle the partial observability inherent in the state space, X-MOBILITY maintains a belief over possible states, approximated using a parameterized Normal distribution $s \sim \mathcal{N}(\mu_\theta, \sigma_\theta)$, where μ_θ and σ_θ denote the mean and covariance, respectively. These statistics are continuously estimated and updated by the state estimation network, ensuring the belief remains current as new observations are made.

C. State Estimation Network

Classical Bayesian state estimation is impractical in the learned state space due to unknown motion and observation models, as well as the potential for non-Markovian effects. To address these challenges, X-MOBILITY introduces a state estimation network, where a history state h_t is used to capture non-Markovian effects, together with a recurrent unit to propagate the history state over time. With the observation embeddings o_t produced by observation encoders and the applied action a_t, the belief state is modeled as:

$$s_t \sim \mathcal{N}(\mu^e_\theta(h_{t-1}, a_{t-1}, o_t), \sigma^e_\theta(h_{t-1}, a_{t-1}, o_t)I), \quad (1)$$

where the recurrent history transition follows:

$$h_t = f_\theta(h_{t_{t-1}}, s_t). \quad (2)$$

The history h_{t-1} and state s_t are then concatenated to form a 1-D latent state $z_t = [h_{t-1}, s_t]$ used for multi-task decoding.

1) State Transition Network: To model these transitions, the f_θ is implemented as a gated recurrent unit (GRU), while $(\mu^e_\theta, \sigma^e_\theta)$ are modeled as multi-layer perceptrons (MLPs). More specifically, within the state transition network as shown in Fig. 2(b), the input action command is first processed through an MLP to produce a higher-dimensional feature state. This feature state is then concatenated with the history and observation embedding to estimate $(\mu_\theta, \sigma_\theta)$ via a MLP-based Normal distribution model. Finally, a sampler draws a state from the learned distribution.

2) Observation Encoding: To update the belief state, X-MOBILITY takes front-view image and robot states, including linear and angular velocities, as inputs. These inputs are embedded into a compact, low-dimensional representation to effectively learn the dynamics described in Eqn. (1). To capture visual features, we employ a pretrained DINOv2 [20] model as the image encoder. The image is embedded into a 1-D token $u_t \in \mathbb{R}^{768}$ by concatenating the class token with the average-pooled patch tokens. DINOv2's strong feature representation and robust performance across various image conditions can enhance X-MOBILITY's ability to generalize

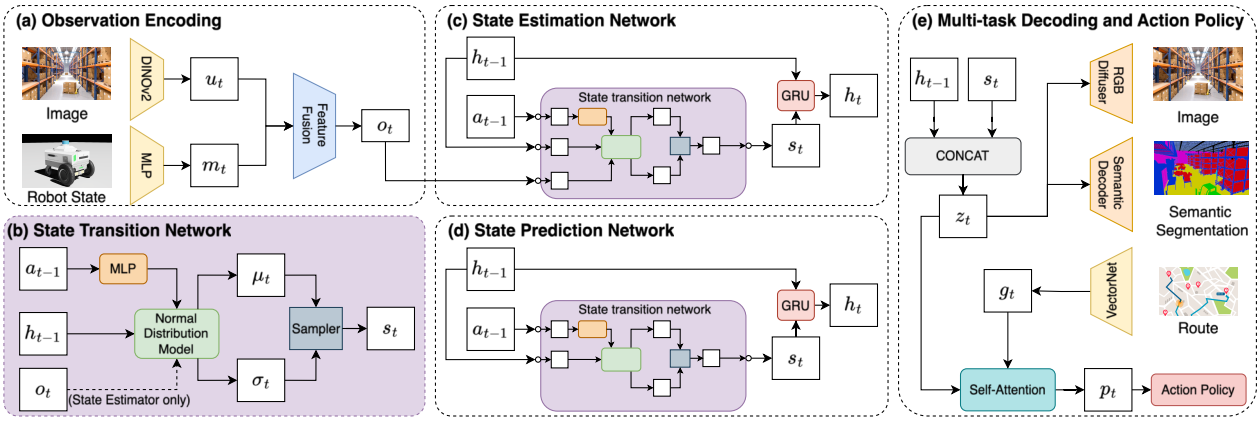

Fig. 2. Model architecture: i) The state estimator in (c) and state predictor in (d) are designed to capture world dynamics; ii) Along with the multi-task decoders in (e), they generate a rich latent space representation for action policy learning, which takes the latent state and route embedding as inputs.

well across different environments in both simulation and real world. The linear and angular speeds are encoded into $m_t \in \mathbb{R}^{32}$ using fully connected layers. This speed embedding is then concatenated with the image embedding to produce the observation embedding $o_t = [u_t, m_t]$ for state estimation.

3) Multi-task Decoding: Making sure the latent state z_t captures necessary information is essential for the downstream navigation skill training, which requires well-designed decoders. In X-MOBILITY, we adopt two decoding heads: RGB reconstruction and semantic segmentation as in Fig. 2(e). For RGB reconstruction, we fine-tuned a latent diffusion model (LDM) [21], where the latent state z_t conditions the UNet during the denoising process, following a similar approach as in [22]. The difference between the imposed and predicted noises is penalized via a MSE loss. Semantic segmentation is conducted in a perspective view to more accurately capture stacked obstacles that are not well represented in a bird's-eye view, a method widely used in the autonomous driving domain [23]. To achieve this, we utilize StyleGAN [24] to decode the semantic segmentation from the latent state, starting with a learned constant tensor that is progressively upsampled to the final resolution. At each upsample resolution, we compute the Cross-Entropy loss for the predicted semantics .

By training the encoders, decoders, and state estimation network together end-to-end, X-MOBILITY can produce a rich latent state that effectively supports action policy learning.

D. State Prediction Network

To support applications such as long-horizon planning, which require predicting sequences of planned actions, X-MOBILITY also introduces a state prediction network that forecasts the consequences of future actions. Without using observation embeddings, the belief state in the state predictor is modeled as:

$$s_t \sim \mathcal{N}(\mu_\theta^p(h_{t-1}, a_{t-1}), \sigma_\theta^p(h_{t-1}, a_{t-1})I). \quad (3)$$

The state prediction network is implemented similarly to the state estimation network, with a GRU for recurrent history updates and a Normal distribution model for the state, but without observation embeddings as input.

For state predictor training, a Kullback-Leibler divergence loss is applied between the state estimator and state predictor's belief distributions, encouraging the state predictor to match the state estimator. This allows the model to predict future states that match observed data, improving navigation performance in dynamic environments and enabling broader applications of the world model in various contexts [17].

E. Action Policy Network

With the world modeling in place, the latent state can provide a rich representation of both the environment and the robot's state. The action policy network, depicted in Fig. 2(e), leverages this latent state z_t, along with the route encoding g_t, to predict the action commands $a_t \sim \pi(a_t|z_t, g_t)$.

1) Route Encoding: The route provides essential high-level guidance for navigation, particularly in large-scale environments. While robot localization is required for X-MOBILITY, the route can be as simple as a straight line from the robot's position to its destination, or it can consist of a sequence of waypoints from the environment's topological graph, if available. This design enables X-MOBILITY to function effectively in large environments while avoiding the costly route search used in [1].

To digest route information, we first transform the global route into the robot's frame, allowing the model to operate without requiring a map. We then truncate the regional route segment near the robot, obtaining a series of route poses with x and y positions. This segment is encoded using VectorNet [25], which produces a 1-D feature embedding $g \in \mathbb{R}^{64}$. Compared to rasterizing the route as an image [15], VectorNet can more effectively capture route details and offer the flexibility to encode additional attributes, such as the final destination flag, which are critical for robot navigation.

2) Action Decoding: To decode action policies, the latent state and route embedding are fused using a self-attention mechanism, allowing the model to balance route adherence with environmental interaction. The fused policy state p_t is then decoded by an MLP to generate action commands $a \in$

$\mathbb{R}^6$, representing the desired linear and angular velocities in the x, y, and z directions, along with an optional navigation path $p \in \mathbb{R}^{5\times2}$, which includes five path poses in the robot frame. The action policy head is trained using an $L1$ loss to imitate the teacher policy. For state estimation or prediction, only the action commands are used, while the imitated path can better shape the latent states and support post-process tasks like safety checking.

F. Multi-stage Training

As illustrated in Fig. 2, the action a_t can originate either from the learned policy or from the dataset. This flexibility allows for a multi-stage training pipeline. In the first stage, we deactivate the action policy network and use logged actions from the off-policy dataset to train the world model. Once the world model is sufficiently trained, we activate the action policy network and train it alongside the world model using the on-policy dataset in the second stage. This approach efficiently utilizes large-scale datasets that lack high-quality teacher policies and allows the model to fully explore world dynamics. This is one of the key factors that enables X-MOBILITY to generalize effectively across out-of-distribution environments not present in the training dataset.

III. EXPERIMENT SETTING

A. Dataset

The training data was collected using the Nova Carter robot in Isaac Sim. This photorealistic synthetic dataset includes two types of action policy inputs: random actions and a teacher policy based on Nav2 [1]. For the random action dataset, a customized Synthetic Data Generation (SDG) pipeline was built in Isaac Sim. During data collection, robot initial positions were randomly sampled first, and random actions were applied for several steps before switching to a new set of random actions. A carefully designed randomization logic was implemented to ensure comprehensive state-action coverage, allowing sufficient exploration of the environment. For the teacher policy dataset, we integrated Nav2 into this pipeline for a closed-loop simulation, with navigation tasks generated by randomly sampling start and goal pairs.

Data was recorded at 5Hz across four distinct warehouse environments, each featuring randomly generated textures and layouts. Lighting diversity was also maintained to be as realistic as possible, as shown in Fig. 3. Warehouses were chosen because they provide large-scale, semi-structured, and cluttered settings, offering a diverse range of scenarios for the dataset. In total, we gathered 100K frames from the Nav2 policy and 160K frames from the random action policy, which are evenly distributed across the four scenarios. Each frame includes key fields as outlined in Table I. The *image* field contains the RGB input from the front-view camera, while the *semantic label* field identifies each pixel according to the predefined semantic classes: [*Navigable, Forklift, Cone, Sign, Pallet, Fence, Background*], which is to cover critical object types for safe navigation in warehouse environments. The *route* field contains the regional route segment transformed into the robot's frame, and the

TABLE I
DATASET ELEMENTS

Field	Shape	Teacher	Random Action
image	$\mathbb{R}^{3\times320\times512}$	✓	✓
speed	$\mathbb{R}^2$	✓	✓
semantic label	$\mathbb{R}^{7\times320\times512}$	✓	✓
route	$\mathbb{R}^{20\times2}$	✓	×
path	$\mathbb{R}^{5\times2}$	✓	×
action command	$\mathbb{R}^6$	✓	✓

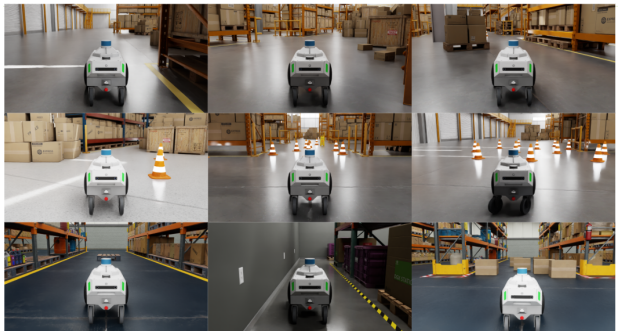

Fig. 3. Benchmark warehouse scenarios with varying levels of difficulty.

speed field captures the robot's linear and angular velocities. The learning targets, *path* and *action command*, include a sequence of path poses and the desired linear and angular velocities in the x, y, and z dimensions.

B. Training

The world model was first trained for 100 epochs using the random action dataset, with a batch size of 32, distributed across 8 H100 GPUs. Each training sample contained 5 frames of data to facilitate world dynamics learning in an auto-regressive manner. Following this, the action policy was trained alongside the world model for an additional 100 epochs using the teacher policy dataset. The RGB diffuser was disabled in the action policy stage to improve training speed. The AdamW optimizer was employed with a learning rate of 10^{-5}.

C. Evaluation

The model was evaluated in both open-loop and closed-loop settings. For the open-loop evaluation, we measured the Mean Absolute Error (MAE) for linear speed (L-MAE), angular speed (A-MAE), and path prediction (P-MAE) using the test split of the training dataset. In the closed-loop evaluation, we built a navigation benchmark suite with 10 warehouse scenarios of varying difficulty, including narrow corridors, cluttered spaces, and low-lying obstacles (see Fig. 3). To assess the model's performance in out-of-distribution cases, 5 of these warehouse scenarios were set in environments not included in the training data. Additionally, 50 randomly generated cluttered scenes in narrow corridors were created to systematically evaluate generalization capability. Each method was tested over 100 runs by conducting 5 trials per designed warehouse scenario. We tracked three key metrics: 1) mission success rate (SR), 2) weighted trip time (WTT), which measures navigation efficiency by dividing

TABLE II
OPEN LOOP METRICS

Method	A-MAE ↓	L-MAE ↓	P-MAE ↓
MILE	0.0751	0.0106	0.1192
BC	0.0870	0.0165	0.1176
X-MOBILITY	0.0747	0.0158	0.1156
X-MOBILITY w/ DP*	0.0306	0.1115	0.0389

* DP: Diffusion Policy

trip time by success rate, and 3) average absolute angular acceleration (AA) as an indicator of motion smoothness.

IV. RESULTS

A. Navigation Performance

We compared X-MOBILITY's navigation performance against several baselines: its teacher policy Nav2, a model-free behavior cloning (BC) method, and a prior state-of-the-art model-based imitation learning method, MILE [15]. The BC method used the same observation encoder as X-MOBILITY but directly mapped observations to policies. For MILE, we adapted the model to predict semantic information in a perspective view rather than a bird's-eye view (BEV), as originally proposed, to better align with robotic applications. All methods were retrained on the same dataset to ensure a fair comparison.

As shown in Table II, all learning-based methods were properly trained and achieved similar open-loop metrics on the test dataset. In the close loop evaluation, X-MOBILITY consistently outperforms the other methods across all navigation metrics, as detailed in Table III. Compared to it's teacher Nav2, X-MOBILITY achieves higher mission success rates and smoother motion in every scenario. In tests involving narrow corridors and multiple homotopy classes, Nav2 often fails or exhibits indecision, exposing common limitations of classical approaches. In contrast, X-MOBILITY successfully navigates these challenges by reasoning through key navigation skills learned from demonstrations and producing smoother motion via leveraging its ability of retaining a history of actions and observations. Behavior Cloning (BC), which lacks the world dynamics understanding and history tracking, results in less smooth trajectories and lower success rates, particularly in out-of-distribution random obstacle scenarios. MILE performs the worst, with the robot struggling to follow the designated route, likely due to route information loss during encoding and feature compression. Additionally, MILE's reliance on ResNet-18 for image encoding, instead of DINOv2, may further contribute to its underperformance.

B. Ablation Studies

1) World Model Pretrain: Pretraining the world model on the 160K frames of random action data leads to improvements in navigation performance, particularly in scenarios involving random obstacles. This demonstrates the model's enhanced ability to manage out-of-distribution cases by leveraging off-policy data. Additionally, semantic segmentation performance improves as well, benefiting from the expanded dataset used for training the semantic decoder, as shown in Table IV.

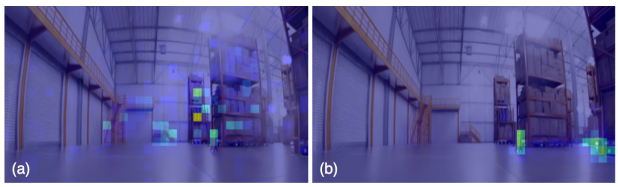

Fig. 4. Attention analysis: (a) pretrained DINOv2, (b) fine tuned with semantic decoding. Attention is directed toward key semantic objects (e.g., sign, pallet, fence) when semantic decoding is enabled, as opposed to being scattered across the entire image without semantic decoding.

2) Semantic Decoding: To assess the impact of semantic decoding on action policy, we compared methods with and without semantic segmentation, as shown in Table III. Without semantic decoding, the model performs better in warehouse scenarios but shows a decline in performance in random obstacle environments, indicating reduced generalizability. This supports our hypothesis that semantic decoding plays a important role in learning world dynamics, ensuring that the latent state contains meaningful environment information for navigation, which is useful for effective policy learning. This conclusion is further reinforced by the attention plots in Fig. 4, where the attention is correctly directed toward key semantic objects when semantic decoder is enabled. The improved performance in warehouse scenarios without semantic decoding suggests possible overfitting to the specific environments in the dataset.

3) History Tracking: To assess the impact of history tracking strategies, we compared the closed-loop performance of our model using two approaches: resetting and fully recurrent. In the resetting approach, the history and latent state are reinitialized at the start of each cycle, while in the recurrent mode, the history and latent state are initialized only once at the beginning and continuously updated with new observations. The recurrent mode showed superior performance, resulting in much smoother motion. In contrast, the resetting mode led to issues such as the robot frequently overshooting during turns. These results indicate that the model effectively learns a representation capable of integrating history over longer horizons, beyond those encountered during training.

4) Diffusion Policy: To enhance the action policy, we also experimented with the diffusion policy [26], conditioning on policy state p_t to jointly decode action commands and paths. The diffusion policy produced mixed results overall. While path prediction is improved, the linear action commands degraded notably, as reflected in the open loop metrics in Table II. In closed-loop evaluation, the diffusion policy struggled to provide consistent action commands and had difficulty in navigating to the destination without collisions. A possible explanation is that the model generates single-step commands instead of predicting a sequence, leading to a low signal-to-noise ratio during the denoising process, then causes instability. Further investigation into this issue is planned as future work.

TABLE III

CLOSE LOOP NAVIGATION BENCHMARK

Method	Warehouse Scenarios SR(%) ↑	WTT(s) ↓	AA(rad/s^2) ↓	Random Obstacle Scenarios SR(%) ↑	WTT(s) ↓	AA(rad/s^2) ↓
ROS Nav2	58	68.41	0.606	34	74.94	0.391
MILE	34	128.32	0.274	26	113.49	0.212
BC	92	40.4	0.219	56	42.72	0.219
X-MOBILITY	**96**	**37.7**	**0.206**	**68**	**40.21**	**0.186**
Ablation Studies						
X-MOBILITY w/o Pretrain	88	43.97	0.215	44	65.45	0.213
X-MOBILITY w/o Semantic	**96**	**33.7**	**0.203**	36	77.77	0.273
X-MOBILITY w/o History Tracking	72	37.5	0.264	16	175.10	0.229
X-MOBILITY	**96**	37.7	0.206	**68**	**40.21**	**0.186**

TABLE IV

SEMANTIC SEGMENTATION IOU

Method	Navigable	Forklift	Cone	Pallet	Fence	Sign
MILE	0.96	0.08	0.09	0.26	0.19	0.07
w/o Pretrain	0.97	0.13	0.14	0.38	0.27	0.12
X-MOBILITY	**0.97**	**0.16**	**0.18**	**0.42**	**0.32**	**0.15**

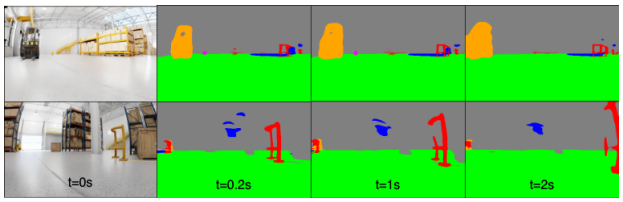

Fig. 5. Qualitative examples of prediction by decoding semantic segmentation from latent state. Green: Navigable, Red: Fence, Blue: Pallet, Orange: Forklift, Purple: Sign

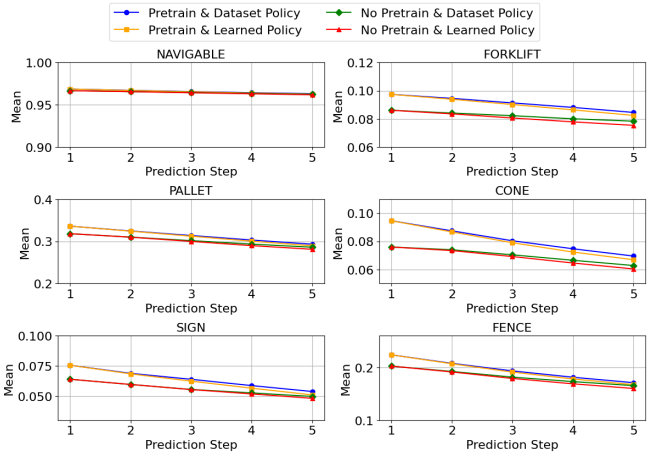

Fig. 6. IOU for semantic decoding at predicted steps.

TABLE V

INFERENCE LATENCY AND GPU MEMORY USAGE

Inference Mode	P50(ms)	P95(ms)	GPU Memory(MB)
Policy Only	38.6	41.96	594
Policy + Semantic	55.55	55.72	804

C. World State Prediction

With state predictor, X-MOBILITY can predict future states in the latent space, which are then decoded into semantic segmentation for better interpretability, as shown in Fig. 5. To evaluate the quality of these predictions, we calculated the Intersection over Union (IOU) for the predicted semantic segmentations over 5 predicted steps. The results, shown in Fig. 6, compare X-MOBILITY with and without pretraining, and with or without the learned policy enabled for prediction. Across all setups, the model can consistently detect and predict the navigable surface, a key element for successful navigation. X-MOBILITY without pretraining shows lower IOU across all semantic classes compared to its pretrained counterpart. As expected, enabling the learned action policy introduces more prediction error as the prediction horizon extends.

V. DISCUSSIONS

A. Sim2Real Transfer

We successfully deployed X-MOBILITY on a Nova Carter robot without any fine-tuning, demonstrating its navigation capabilities through zero-shot Sim2Real transfer. The robot was able to safely navigate through cluttered obstacles in a lab environment (see Fig. 1), despite the environment being absent from the training dataset, further showcasing its ability to generalize to out-of-distribution scenarios.

We believe that several factors contribute to the zero-shot Sim2Real transferability. First, the DINOv2 encoder offers strong feature extraction and representation for input images under various conditions. Second, the probabilistic world model design can enhance the system robustness, particularly against dynamic noises from the real robot, which cannot be fully captured during simulation. Lastly, the synthetic dataset collected using Isaac Sim's Digital Twin is highly photorealistic, further enabling smooth Sim2Real transfer.

Additionally, we also benchmarked the model's inference time on the Jetson AGX Orin, as showing in Table V, underscoring X-MOBILITY's high computational efficiency for edge-device navigation.

B. Cross Embodiment

To further evaluate X-MOBILITY's generalization capability, we also tested its performance across different embodiments. Thanks to its end-to-end design and standardized

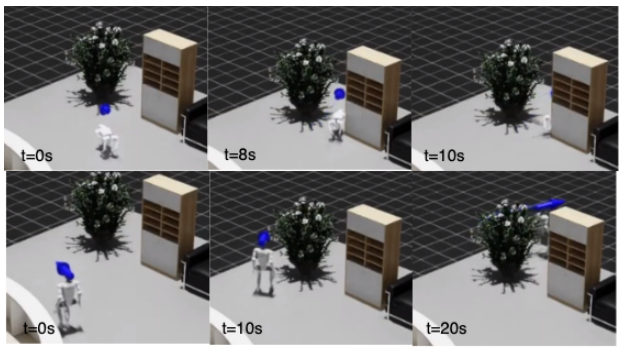

Fig. 7. X-MOBILITY on humanoid and quadruped robots with RL-based locomotion policy pre-trained in Isaac Lab.

input and output formats, we successfully deployed X-MOBILITY on four robot platforms in Isaac Sim: Nova Carter (differential drive), Forklift (Ackermann drive), Unitree Go2 (quadruped), and Unitree G1 (humanoid). By integrating X-MOBILITY with the appropriate controllers or RL locomotion policies pre-trained in Isaac Lab [27], each robot was able to navigate successfully (see Fig. 7). While navigation performance varied depending on camera placement and the robots' dynamic constraints, X-MOBILITY consistently demonstrated good potential for generalization across different embodiments.

VI. FUTURE WORK

In this work, we proposed X-MOBILITY, a generalizable navigation model that integrates world modeling with imitation learning to enhance navigation performance and generalization across out-of-distribution scenarios.

Looking ahead, future work will focus on further enhancing X-MOBILITY's cross-embodiment capability. This will involve incorporating more detailed robot specification encoding to better adapt the model to different platforms. Additionally, we plan to leverage RL fine-tuning to refine the model's performance across various embodiments. Another key direction will be to enrich the dataset with more diverse scenes with dynamic obstacles, enabling a deeper exploration of the world model's role in supporting action policy learning. These advancements will help in solidifying X-MOBILITY as a robust and adaptable solution for navigation across a wide range of environments and robot types.

REFERENCES

[1] S. Macenski, F. Martín, R. White, and J. Ginés Clavero, "The Marathon 2: A Navigation System," in *2020 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2020. [Online]. Available: https://github.com/ros-planning/navigation2

[2] W. Liu, Z. Weng, Z. Chong, X. Shen, S. Pendleton, B. Qin, G. M. J. Fu, and M. H. Ang, "Autonomous vehicle planning system design under perception limitation in pedestrian environment," in *2015 IEEE 7th International Conference on Cybernetics and Intelligent Systems (CIS) and IEEE Conference on Robotics, Automation and Mechatronics (RAM)*. IEEE, 2015, pp. 159–166.

[3] C. Mavrogiannis, F. Baldini, A. Wang, D. Zhao, P. Trautman, A. Steinfeld, and J. Oh, "Core challenges of social robot navigation: A survey," *ACM Transactions on Human-Robot Interaction*, vol. 12, no. 3, pp. 1–39, 2023.

[4] M. T. Spaan, "Partially observable Markov decision processes," in *Reinforcement learning: State-of-the-art*. Springer, 2012, pp. 387–414.

[5] W. Liu, S.-W. Kim, S. Pendleton, and M. H. Ang, "Situation-aware decision making for autonomous driving on urban road using online POMDP," in *2015 IEEE Intelligent Vehicles Symposium (IV)*. IEEE, 2015, pp. 1126–1133.

[6] A. Somani, N. Ye, D. Hsu, and W. S. Lee, "DESPOT: Online POMDP planning with regularization," *Advances in neural information processing systems*, vol. 26, 2013.

[7] P. Mirowski, R. Pascanu, F. Viola, H. Soyer, A. J. Ballard, A. Banino, M. Denil, R. Goroshin, L. Sifre, K. Kavukcuoglu *et al.*, "Learning to navigate in complex environments," *arXiv preprint arXiv:1611.03673*, 2016.

[8] R. Doshi, H. Walke, O. Mees, S. Dasari, and S. Levine, "Scaling Cross-Embodied Learning: One Policy for Manipulation, Navigation, Locomotion and Aviation," *arXiv preprint arXiv:2408.11812*, 2024.

[9] A. Sridhar, D. Shah, C. Glossop, and S. Levine, "NoMaD: Goal Masked Diffusion Policies for Navigation and Exploration," *arXiv pre-print*, 2023. [Online]. Available: https://arxiv.org/abs/2310.07896

[10] K. Zhu and T. Zhang, "Deep reinforcement learning based mobile robot navigation: A review," *Tsinghua Science and Technology*, vol. 26, no. 5, pp. 674–691, 2021.

[11] L. Qin, Z. Huang, C. Zhang, H. Guo, M. Ang, and D. Rus, "Deep imitation learning for autonomous navigation in dynamic pedestrian environments," in *2021 IEEE International Conference on Robotics and Automation (ICRA)*. IEEE, 2021, pp. 4108–4115.

[12] M. Zare, P. M. Kebria, A. Khosravi, and S. Nahavandi, "A survey of imitation learning: Algorithms, recent developments, and challenges," *IEEE Transactions on Cybernetics*, 2024.

[13] S. Ross, G. Gordon, and D. Bagnell, "A reduction of imitation learning and structured prediction to no-regret online learning," in *Proceedings of the fourteenth international conference on artificial intelligence and statistics*. JMLR Workshop and Conference Proceedings, 2011, pp. 627–635.

[14] D. Ha and J. Schmidhuber, "World models," *arXiv preprint arXiv:1803.10122*, 2018.

[15] A. Hu, G. Corrado, N. Griffiths, Z. Murez, C. Gurau, H. Yeo, A. Kendall, R. Cipolla, and J. Shotton, "Model-based imitation learning for urban driving," *Advances in Neural Information Processing Systems*, vol. 35, pp. 20 703–20 716, 2022.

[16] F. Jia, W. Mao, Y. Liu, Y. Zhao, Y. Wen, C. Zhang, X. Zhang, and T. Wang, "ADriver-i: A general world model for autonomous driving," *arXiv preprint arXiv:2311.13549*, 2023.

[17] Y. Wang, J. He, L. Fan, H. Li, Y. Chen, and Z. Zhang, "Driving into the Future: Multiview Visual Forecasting and Planning with World Model for Autonomous Driving," *arXiv preprint arXiv:2311.17918*, 2023.

[18] J. Crespo, J. C. Castillo, O. M. Mozos, and R. Barber, "Semantic information for robot navigation: A survey," *Applied Sciences*, vol. 10, no. 2, p. 497, 2020.

[19] X. Xiao, J. Biswas, and P. Stone, "Learning inverse kinodynamics for accurate high-speed off-road navigation on unstructured terrain," *IEEE Robotics and Automation Letters*, vol. 6, no. 3, pp. 6054–6060, 2021.

[20] M. Oquab, T. Darcet, T. Moutakanni, H. V. Vo, M. Szafraniec, V. Khalidov, P. Fernandez, D. Haziza, F. Massa, A. El-Nouby, R. Howes, P.-Y. Huang, H. Xu, V. Sharma, S.-W. Li, W. Galuba, M. Rabbat, M. Assran, N. Ballas, G. Synnaeve, I. Misra, H. Jegou, J. Mairal, P. Labatut, A. Joulin, and P. Bojanowski, "DINOv2: Learning Robust Visual Features without Supervision," 2023.

[21] R. Rombach, A. Blattmann, D. Lorenz, P. Esser, and B. Ommer, "High-Resolution Image Synthesis with Latent Diffusion Models," 2021.

[22] X. Wang, Z. Zhu, G. Huang, X. Chen, J. Zhu, and J. Lu, "Drive-Dreamer: Towards Real-world-driven World Models for Autonomous Driving," *arXiv preprint arXiv:2309.09777*, 2023.

[23] C. Yang, Y. Chen, H. Tian, C. Tao, X. Zhu, Z. Zhang, G. Huang, H. Li, Y. Qiao, L. Lu, J. Zhou, and J. Dai, "BEVFormer v2: Adapting Modern Image Backbones to Bird's-Eye-View Recognition via Perspective Supervision," *ArXiv*, 2022.

[24] T. Karras, S. Laine, and T. Aila, "A style-based generator architecture for generative adversarial networks," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2019, pp. 4401–4410.

[25] J. Gao, C. Sun, H. Zhao, Y. Shen, D. Anguelov, C. Li, and C. Schmid, "Vectornet: Encoding hd maps and agent dynamics from vectorized representation," in *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 2020, pp. 11 525–11 533.
[26] C. Chi, Z. Xu, S. Feng, E. Cousineau, Y. Du, B. Burchfiel, R. Tedrake, and S. Song, "Diffusion Policy: Visuomotor Policy Learning via Action Diffusion," *The International Journal of Robotics Research*, 2024.
[27] M. Mittal, C. Yu, Q. Yu, J. Liu, N. Rudin, D. Hoeller, J. L. Yuan, R. Singh, Y. Guo, H. Mazhar, A. Mandlekar, B. Babich, G. State, M. Hutter, and A. Garg, "Orbit: A Unified Simulation Framework for Interactive Robot Learning Environments," *IEEE Robotics and Automation Letters*, vol. 8, no. 6, pp. 3740–3747, 2023.

APPENDIX I
MODEL DESCRIPTION

To provide a more detailed overview of the X-MOBILITY model, the model parameters are listed in Table VI, while Table VII outlines the hyperparameters used for model training.

TABLE VI
MODEL PARAMETERS

	Name	Parameters
Encoders	Image encoder	22.1M
	Robot state encoder	1.1K
State Estimators	State estimator	5.5M
	State predictor	2.3M
Action Policy	Route encoder	8.5K
	Self-attention fuser	28.5M
	Action command decoder	10.5M
	Path decoder	10.5M
Decoders	Semantic decoder	34.2M
	RGB diffuser	962M

TABLE VII
HYPERPARAMETERS

Category	Name	Value
Training	GPUs	8 H100
	Batch size	32
	Precision	16-mixed
	World model epoches	100
	Action policy epoches	100
Optimizer	Type	AdamW
	Learning rate	1e-5
	Weight decay	0.01
	Scheduler	OneCycleLR
	Scheduler pct start	0.2
Observation Encoders	Image size	320×512
	Image embedding dim	768
	Robot state embedding dim	32
State Estimation	History dim	1024
	State dim	512
	Action encoding dim	64
Action Policy	VectorNet layers	4
	Policy state dim	2048
Semantic Decoder	StyleGan constant size	(5, 8)
RGB Diffuser	Noise scheduler	LMS discrete
	Beta schedule	scaled linear
	Beta start	0.00085
	Beta end	0.012
	Training timesteps	1000
	Inference timesteps	50
Losses	Action command weight	10.0
	Path weight	5.0
	Semantic weight	1.0
	RGB weight	10.0
	KL weight	0.001
	KL balancing alpha	0.75